# Session 24 — Study Guide

Status: PREPARED DRAFT

## What we learned today

1. **Tailwind utility-first CSS** — replacing plain CSS/CSS Modules with utility classes directly in JSX, scanning only the real files listed in `content`.
2. **A Client Component driving real URL search params** — `useSearchParams` / `usePathname` / `useRouter` from `next/navigation`, with the URL itself as the real shared state between a Client Component and the Server Component rendering around it.
3. **Sending the real backend's own query filters** — `status` and `title`, never reinventing client-side filtering the backend already supports.
4. **React Context for app-wide state** — a real light/dark theme toggle shared across every page, with a custom hook that fails loudly outside its provider.

## Why we needed it

Session 23 built a working, but completely unstyled, routing skeleton. Every real page rendered real backend data with zero visual design and zero interactivity beyond links. Today gives the app a real, consistent visual language, and introduces the first real Client Component — the piece every future interactive feature (search, sort, filters, theme, eventually mutations) will build on.

## First-Time Tool Box

**Tool:** Tailwind CSS v3 (this cohort's first real exposure).

**What it is:** a utility-first CSS framework — styling is applied through small, composable classes directly in JSX, instead of writing separate CSS rules.

**Why we use it here:** it is pinned to v3, not the newest major version, because v4's own Node.js ESM/`magic-string` interop genuinely breaks under this program's pinned Node 18.20.8 + Turbopack — confirmed by a real, reproduced build failure, not assumed (see homework STRETCH).

**Where/how you run it:** configured once in `tailwind.config.js` / `postcss.config.js`; after that, just write `className="..."` directly in any component.

**Commands/actions needed today:**

1. `npm install -D tailwindcss@3.4.17 postcss@8.5.1 autoprefixer@10.4.20`
2. Creating `tailwind.config.js` (`darkMode: "class"`, `content`) and `postcss.config.js`.
3. Adding the three `@tailwind` directives to `globals.css`.
4. `npx tsc --noEmit` / `npm run build` to verify.

**How to tell it worked:** utility classes (e.g. `bg-gray-50`, `dark:bg-gray-950`) visibly change the rendered page.

**Common first-time mistakes:** forgetting `darkMode: "class"` (the theme toggle would then do nothing); listing the wrong `content` path (no classes ever generate).

## The problem, before the solution

Before today, every page rendered real backend data correctly, but with zero visual design and zero way for a user to filter, search, or control anything — Session 23 deliberately stopped at "it works," not "it's usable."

## The solution, step by step

**Utility-first styling with Tailwind.** WHAT IS IT: composable classes like `flex`, `gap-4`, `dark:text-gray-100`, applied directly in JSX. WHY: no separate CSS file to keep in sync with the component; `dark:` variants make theme-aware styling a single extra class, not a second stylesheet. WHAT BREAKS WITHOUT IT: nothing breaks, but every page stays visually raw, like Session 23 left it. HOW: `className="flex flex-col gap-3 dark:bg-gray-950"`. WHAT TO LOOK AT: `src/components/ui/{Button,Card,Badge}.tsx`. WHAT TO EXPECT: a consistent, styled app, with the SAME few primitives reused everywhere.

**A Client Component driving the real URL.** WHAT IS IT: `"use client"` + `useSearchParams` / `useRouter` reading and writing the URL's own query string. WHY: the Server Component above it (Project Detail) re-reads that same URL on the next request and re-fetches from the real backend — the URL is the shared state, not React state. WHAT BREAKS WITHOUT IT: using `useState` instead would create a second source of truth that can drift from what the Server Component actually fetched. HOW: `router.push(` ${pathname}?${params.toString()} `)`. WHAT TO LOOK AT: `src/components/issues/StatusFilter.tsx`. WHAT TO EXPECT: changing a filter updates the URL, and the page re-fetches filtered backend data.

**React Context for the theme toggle.** WHAT IS IT: `createContext` / `useContext`, holding one real piece of state (the theme) shared app-wide. WHY: avoids passing `theme` / `toggleTheme` down through every layout and page as props. WHAT BREAKS WITHOUT IT: `useTheme()` called outside `ThemeProvider` throws a specific, loud error — by design, so a missing provider is caught immediately, not later as a confusing crash. HOW: `const { theme, toggleTheme } = useTheme();` WHAT TO LOOK AT: `src/context/ThemeContext.tsx`, `src/components/ui/ThemeToggle.tsx`. WHAT TO EXPECT: clicking the toggle switches the whole app between light and dark instantly.

## Connecting to earlier sessions

Context itself is not new — PowerUp already covered `createContext` / `useContext`; what's new is using it for a real app-wide concern (theme) inside the App Router's own Server/Client Component split, where the provider boundary must start at a specific, deliberate point in the tree, not necessarily the root.

## Glossary (required)

- **Utility-first CSS:** styling through small, single-purpose classes composed directly in markup, instead of writing custom CSS rules per component.
- **`darkMode: "class"`:** Tailwind's own dark-mode strategy where `dark:` variants activate only when a `dark` class is explicitly present on an ancestor element (usually `<html>`).

- **Client Component:** a component marked `"use client"`, running in the browser — required for `useState`, `useEffect`, event handlers, and any `next/navigation` client hook.
- **React Context:** a mechanism for sharing one value across a component tree without passing it through every level as a prop.

## Common mistakes

- Reaching for `useState` to track a filter value that the URL already holds.
- `router.replace` instead of `router.push` — silently breaks the browser back button.
- Forgetting `darkMode: "class"` in `tailwind.config.js`.
- A combined `useEffect` that both reads the OS preference and applies the theme class — overwrites a user's own explicit choice on every toggle.

## Common errors and how to debug them

- **Tailwind classes have no visible effect:** check `tailwind.config.js`'s own `content` array actually includes the file you're editing, and that the three `@tailwind` directives are present in `globals.css`.
- **The theme toggle does nothing:** confirm `darkMode: "class"` is set, and that `ThemeProvider` actually wraps the part of the tree you're looking at.
- **`useTheme must be used inside a ThemeProvider`:** a real component is calling `useTheme()` from outside the provider's own subtree — move the component, or move the provider up.

Never "fix" any of these by removing the error and returning a fake default theme — the error exists specifically to catch a real setup mistake immediately.

## Review questions (required)

1. Why does `StatusFilter` hold no `useState` for "the currently selected status" at all?
2. What real, specific problem does `darkMode: "class"` solve that the default `"media"` mode cannot?
3. Why does `ThemeProvider` need two separate `useEffect` calls instead of one?
4. Why does this session pin Tailwind to v3 instead of installing the newest version?

## Answers

1. The URL itself already holds that value; a separate `useState` would be a second, driftable source of truth for the same information the Server Component above it also needs to read.
2. `"media"` only ever follows the OS preference, with no way for a button inside the app to override it; `"class"` lets a real user choice control the same `dark:` utilities instead.
3. One effect must run exactly once (reading the OS preference as a starting point); the other must run every time `theme` changes (applying the class). Merging them would either re-read the OS preference on every toggle, or never apply the class at all.
4. Tailwind v4's own Node.js ESM/`magic-string` interop genuinely fails to build under this program's pinned Node 18.20.8 + Turbopack — a real, reproduced, confirmed incompatibility, the same reliability principle behind every other version pin in this program.

## What you should be able to explain now (required)

- The real difference between styling with Tailwind utilities and writing plain CSS, and why this program chose the former.
- Why a Client Component can be the real source of truth for a piece of UI state (the URL) that a Server Component also depends on, and how the two stay in sync.
- When and why a React Context provider's own boundary should start partway through a component tree, rather than always at the root.

## Not yet (required)

- Server Actions, real mutations (Session 25).
- Redux Toolkit (Session 25).
- Real frontend authentication/tokens (Session 25).
- Persisting the theme choice across a reload — deliberately left open this session; closed by this session's own homework CHALLENGE.